

BÁO CÁO THỰC HÀNH

Thực hành Nhập môn Mạng máy tính

Lab 03: Hàm băm và chữ kí số

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: 21/04/2026

Nhóm 13 - NT101.Q22.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
23520536	Phạm Cao Minh Hoàng	Câu 4	50%
24521388	Nguyễn Hoàng Phúc	Câu 2, câu 3	50%

1. Nhiệm vụ 2.2

2. Nhiệm vụ 2.2

a) Để nhận biết số byte khác nhau giữa 2 thông điệp cho sẵn, thực hiện chia chuỗi thành từng cặp 2 kí tự và tiến hành so sánh:

6 removals	4 lines Copy	6 additions	4 lines Copy
1 d131dd02c5e6eec4693d9a0698aff95c2fca58712467eab4004583eb8fb7f8955ad3	Copy	1 d131dd02c5e6eec4693d9a0698aff95c2fca58712467eab4004583eb8fb7f8955ad3	
2 40609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd7280373c5bd8823e315		2 40609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd7280373c5bd8823e315	
3 6348f5bae6dacd436c919c6dd53e2b487da03fd02396306d248cda0e99f33420f577		3 6348f5bae6dacd436c919c6dd53e2b487da03fd02396306d248cda0e99f33420f577	
4 ee8ce54b67080a80d1ec69821bcb6a8839396f9652b6ff72a70		4 ee8ce54b67080280d1ec69821bcb6a8839396f9652b6ff72a70	

- 87 -> 07
- 71 -> f1
- df -> d7
- b4 -> 34
- a8 -> 28
- ab -> ab
- 6 cặp khác nhau tương ứng với 6 byte khác biệt.
- Để tạo giá trị băm MD5, em viết chương trình python để thực hiện việc xóa dấu xuống dòng cùng khoảng trắng, ép chuỗi văn bản Hex này thành dữ liệu byte thô rồi tính băm MD5, đồng thời xác nhận sau khi so sánh kết quả.

```

import hashlib

-
-
text_msg1 =
"""d131dd02c5e6eec4693d9a0698aff95c2fca58712467eab4004583eb8fb7f8
955ad3
40609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd7280373c5bd8823
e315
6348f5bae6dacd436c919c6dd53e2b487da03fd02396306d248cda0e99f33420f
577
ee8ce54b67080a80d1ec69821bcb6a8839396f9652b6ff72a70"""
-
-
text_msg2 =
"""d131dd02c5e6eec4693d9a0698aff95c2fca58712467eab4004583eb8fb7f8
955ad3
40609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd7280373c5bd8823
e315
6348f5bae6dacd436c919c6dd53e23487da03fd02396306d248cda0e99f33420f
577e
e8ce54b67080280d1ec69821bcb6a8839396f9652b6ff72a70"""
-
-
# 1. Xóa toàn bộ dấu xuống dòng (\n) và khoảng trắng bằng hàm replace()
hex_msg1 = text_msg1.replace('\n', '').replace(' ', '')
hex_msg2 = text_msg2.replace('\n', '').replace(' ', '')
-
-
# 2. Ép chuỗi văn bản Hex thành Dữ liệu Byte thô
bytes_msg1 = bytes.fromhex(hex_msg1)

```

```

- bytes_msg2 = bytes.fromhex(hex_msg2)
-
- # 3. Tính băm MD5
- hash1 = hashlib.md5(bytes_msg1).hexdigest()
- hash2 = hashlib.md5(bytes_msg2).hexdigest()
-
- print(f"MD5 của Message 1: {hash1}")
- print(f"MD5 của Message 2: {hash2}")
-
- if hash1 == hash2:
-     print("\n=> Hai thông điệp đã tạo ra cùng một mã băm.")

```

- Các thành phần cơ bản:
 - o Sử dụng thư viện hashlib để sử dụng thuật toán băm MD5, sử dụng 2 biến text_msg1 và text_msg2 như các dữ liệu đầu vào
 - o Sau khi chạy cho ra được 2 mã băm giống hệt nhau cùng câu xác nhận.
 - o Khỏi xử lý và so sánh: Thực hiện việc làm sạch chuỗi, chuyển đổi kiểu dữ liệu, tính toán mã băm và đưa ra kết luận kiểm tra va chạm.
- Các bước thực hiện:
 - o B1: Làm sạch dữ liệu gốc: sử dụng các hàm replace để xóa bỏ toàn bộ dấu xuống dòng và khoảng trắng dư thừa trong quá trình copy-paste, gom đoạn HEX thành chuỗi liên tục.
 - o B2: Ép kiểu sang Byte thô: sử dụng hàm byte.fromhex() để chuyển đổi chuỗi văn bản thành dữ liệu nhị phân. Do thuật toán MD5 thao tác trực tiếp trên các byte nên dữ liệu buộc phải đưa về dạng này.
 - o B3: Đưa dữ liệu vừa tạo vào làm hashlib.md5() rồi gọi phương thức .hexdigest() để xuất mã băm dưới dạng Hex để dễ đọc

```

MD5 của Message 1: 79054025255fb1a26e4bc422aef54eb4
MD5 của Message 2: 79054025255fb1a26e4bc422aef54eb4

=> Hai thông điệp đã tạo ra cùng một mã băm.

```

b)

- Sau khi chạy các chương trình trên console sẽ xuất hiện các thông báo như "Hello, world!", v.v và dòng lệnh "enter to quit"

```

pau@pau-VirtualBox:~/Downloads$ ./hello
Hello, world!

(press enter to quit)
pau@pau-VirtualBox:~/Downloads$ ./erase
This program is evil!!!
Erasing hard drive...1Gb...2Gb... just kidding!
Nothing was erased.

(press enter to quit)

```

- Thực hiện tạo giá trị băm cho 2 chương trình qua câu lệnh md5sum, quan sát thấy rằng 2 giá trị này là giống nhau dù cho 2 file này là 2 file thực thi riêng biệt

- Đây là minh chứng cho lỗ hổng MD5 Collision: Hacker có thể tạo ra file chứa mã độc - erase rồi làm giả cấu trúc tập tin sao cho sinh ra mã băm MD5 y hệt như phần mềm thông thường - hello để qua mặt các chương trình diệt virus

```
pau@pau-VirtualBox:~/Downloads$ md5sum hello
da5c61e1edc0f18337e46418e48c1290  hello
pau@pau-VirtualBox:~/Downloads$ md5sum erase
da5c61e1edc0f18337e46418e48c1290  erase
```

c)

- Có sự khác biệt duy nhất ở 2 file, đó là màu sắc của tấm nền: shattered-1 nền đỏ và shattered-2 nền xanh
- Tạo băm SHA-1 bằng lệnh sha1sum, quan sát được kết quả tương tự với trường hợp trước đó của MD5: đều cho ra cùng một mã băm dù khác nội dung

```
38762cf7f55934b34d179ae6a4c80cadccbb7f0a  shattered-1.pdf
pau@pau-VirtualBox:~/Downloads$ sha1sum shattered-2.pdf
38762cf7f55934b34d179ae6a4c80cadccbb7f0a  shattered-2.pdf
pau@pau-VirtualBox:~/Downloads$
```

d)

- Kết luận rút ra được:
 - o Hàm băm MD5 và SHA-1 đã bị phá vỡ hoàn toàn về mặt mật mã học. Không còn đảm bảo được tính chất "hai thông điệp khác nhau phải tạo ra hai mã băm khác nhau".
 - o Kẻ tấn công có thể lợi dụng điểm yếu này để qua mặt các hệ thống kiểm tra tính toàn vẹn. Ví dụ: nhúng mã độc vào một phần mềm hợp pháp hoặc làm giả mạo chữ ký số trên các hợp đồng điện tử mà hệ thống quét mã băm không thể phát hiện ra.

- Lí do:

+Về mặt toán học:

- o Về mặt lý thuyết, độ dài của dữ liệu đầu vào là vô hạn. Tuy nhiên, kết quả đầu ra – mã băm lại có độ dài cố định: MD5 chỉ có 128-bit và SHA-1 có 160-bit. Vì không gian đầu ra là hữu hạn trong khi không gian đầu vào là vô hạn, nên theo nguyên lý toán học, chắc chắn phải có nhiều thông điệp khác nhau cho ra cùng một mã băm. Vấn đề chỉ là việc tìm ra chúng tốn bao nhiêu thời gian.

+Về mặt thuật toán:

- o Cả MD5 và SHA-1 đều sử dụng cấu trúc **Merkle-Damgård**, trong đó dữ liệu được chia thành các khối nhỏ (thường là 64 byte/khối) và đưa qua một **Hàm nén** theo từng vòng. Đầu ra của vòng trước sẽ làm đầu vào cho vòng sau
 - o Các chuyên gia mật mã phát hiện ra rằng họ có thể chen các "**khối dữ liệu va chạm**" (**Collision Blocks**) vào thông điệp. Bằng cách thay đổi cấu trúc bit một cách có chủ đích ở một vài vị trí cực kỳ chính xác (như việc chèn 6 byte trong file msg1 và msg2 bạn vừa làm), sự sai lệch này sẽ tự cộng trừ và "triệt tiêu" lẫn nhau trong các vòng tính toán nội bộ của Hàm nén.
- ⇒ Khi đi đến vòng tính toán cuối cùng, trạng thái bên trong của thuật toán bị đưa về lại giống hệt nhau, từ đó xuất ra cùng một chuỗi mã băm cuối cùng.

3. Nhiệm vụ 2.3

- Câu hỏi 1: Nếu độ dài của tệp tiền tố của bạn không phải là bội số của 64, điều gì sẽ xảy ra?

- o B1: Tạo 1 file văn bản ngắn chứa khoảng 30 ký tự như hình

```
pau@pau-VirtualBox:~/Downloads$ echo -n "UIT_MANGMAYTINH_ANTOANTHONGTIN" > prefix_short.txt
```

- o B2: Cho chạy công cụ và chạy md5collgen

```
pau@pau-VirtualBox:~/Downloads$ ./md5collgen -p prefix_short.txt -o out1.bin out2.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)
```

```
Using output filenames: 'out1.bin' and 'out2.bin'
Using prefixfile: 'prefix_short.txt'
Using initial value: adb18fdea878ba78fdb2d8b38d20ae07
```

```
Generating first block: ...
Generating second block: S01.....
Running time: 3.66877 s
```

```
pau@pau-VirtualBox:~/Downloads$
```

```
pau@pau-VirtualBox:~/Downloads$ diff out1.bin out2.bin
```

```
Binary files out1.bin and out2.bin differ
```

```
pau@pau-VirtualBox:~/Downloads$ md5sum out1.bin
```

```
16f69053999cc38f04c27037337083f9 out1.bin
```

```
pau@pau-VirtualBox:~/Downloads$ md5sum out2.bin
```

```
16f69053999cc38f04c27037337083f9 out2.bin
```

```
pau@pau-VirtualBox:~/Downloads$ xxd out1.bin
```

```
00000000: 5549 545f 4d41 4e47 4d41 5954 494e 485f  UIT_MANGMAYTINH_
00000010: 414e 544f 414e 5448 4f4e 4754 494e 0000  ANTOANTHONGTIN..
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  .....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  .....
00000040: 0cf5 2918 6c9a 55f5 a451 697b 02a5 e2c6  ..).l.U..Qi{....
00000050: c7c5 efdb d999 2fe6 2c12 80fa 0d36 2b90  ....../.,....6+.
00000060: bc00 fbd3 620c f01d 0f20 f03c 01f4 9068  ....b....<...h
00000070: 333b 3699 7a97 ed8a 5cd3 6da0 867e b2ec  3;6.z...\..m..~..
00000080: 578c 809a 3e57 2c8d d0d0 872a 61b8 d106  W...>W,....*a...
00000090: 244e 183b 17c6 3671 cb5a 6e49 ec87 deed  $N.;..6q.ZnI....
000000a0: e6cd 3e1a f411 0452 882a 6c17 b0c8 02ab  ..>....R.*l.....
000000b0: 2232 da11 af3b 7e6e ca50 863d e53e 6967  "2...;~n.P.=.>ig
```

```
pau@pau-VirtualBox:~/Downloads$
```

- Có thể thấy băm md5 của 2 file giống hệt nhau dù câu lệnh diff cho biết nội dung 2 file là khác nhau → Thực hiện ép MD5 Collision thành công.
- Trong phần trả lời sau lệnh xxd: có thể thấy :
 - o Phần đệm: Do thuật toán MD5 xử lý dữ liệu theo từng khối (block) 64 bytes, mà tiền tố gốc (30 bytes) không đủ lấp đầy 1 khối, nên công cụ md5collgen đã tự động chèn thêm 34 bytes rỗng (các byte 00 00) vào ngay sau chuỗi tiền tố để làm tròn cho đủ 64 bytes đầu tiên.
 - o Phần va chạm (Từ vị trí 00000040 trở đi): Bắt đầu từ byte thứ 64 (tương ứng với mã offset 040 ở hệ thập lục phân), công cụ mới bắt đầu chèn các

- Câu hỏi 2: Tạo một tệp tiền tố có chính xác 64 byte, chạy lại công cụ xung đột và xem điều gì xảy ra.
 - o B1:Tạo file mới với nội dung là 64 kí tự 7
 - o B2:Cho chạy công cụ va chạm và các câu lệnh như hướng dẫn:

- Phân tích: qua lệnh xxd ta thấy 4 dòng đầu tiên (từ vị trí offset 00000000 đến 00000030) chứa toàn bộ 64 ký tự tiền tố gốc (các số 7 và 4, tương ứng mã Hex là 37 và 34) khác với câu trước, ở đây không xuất hiện bộ đệm 00 00 nào.

- Giải thích: Thuật toán MD5 chia dữ liệu thành các khối (block) cố định là 64 bytes. Vì file tiền tố gốc của chúng ta đã dài chính xác 64 bytes (vừa khít 1 block), công cụ md5collgen không cần phải chèn thêm dữ liệu rác để làm tròn nữa.
 - Qua lệnh md5sum cũng xác nhận rằng cả hai file out10.bin và out11.bin đều tạo ra chung một mã băm MD5 là 880f8d4f8387446f2be2d62fbf48870e.
- ⇒ Việc tạo va chạm MD5 (Collision) trên cùng một cấu trúc tiền tố 64-byte đã diễn ra thành công

4. Nhiệm vụ 2.4

- B1: Bước 1: Tải về một chứng chỉ từ webserver thật
 - Tải và phân tách chứng chỉ từ Web Server: Sử dụng lệnh openssl s_client để kết nối đến máy chủ www.openssl.org:443 và tải về toàn bộ chuỗi chứng chỉ (Certificate Chain).
 - sử dụng lệnh awk để quét chuỗi -----BEGIN CERTIFICATE----- và tự động tách luồng đầu ra thành 2 file riêng biệt:
 - c0.pem: Chứng chỉ của máy chủ (Server Certificate - Chủ thể cần xác minh).
 - c1.pem: Chứng chỉ của Tổ chức phát hành (Issuer / Intermediate CA - Dùng để xác minh).
 - Lệnh ls -l cho thấy 2 file chứng chỉ đã được tạo thành công.

```

vboxuser@phamcaominhhoang23520536: ~
vboxuser@phamcaominhhoang23520536:~$ openssl s_client -connect www.openssl.org:443 -showcerts </dev/null 2>/dev/null | \
awk '/-----BEGIN CERTIFICATE-----/{i++} i==1{print > "c0.pem"} i==2{print > "c1.pem"}'
vboxuser@phamcaominhhoang23520536:~$ ls -l c0.pem c1.pem
-rw-rw-r-- 1 vboxuser vboxuser 2106 Apr 21 09:04 c0.pem
-rw-rw-r-- 1 vboxuser vboxuser 2077 Apr 21 09:04 c1.pem
vboxuser@phamcaominhhoang23520536:~$
  
```

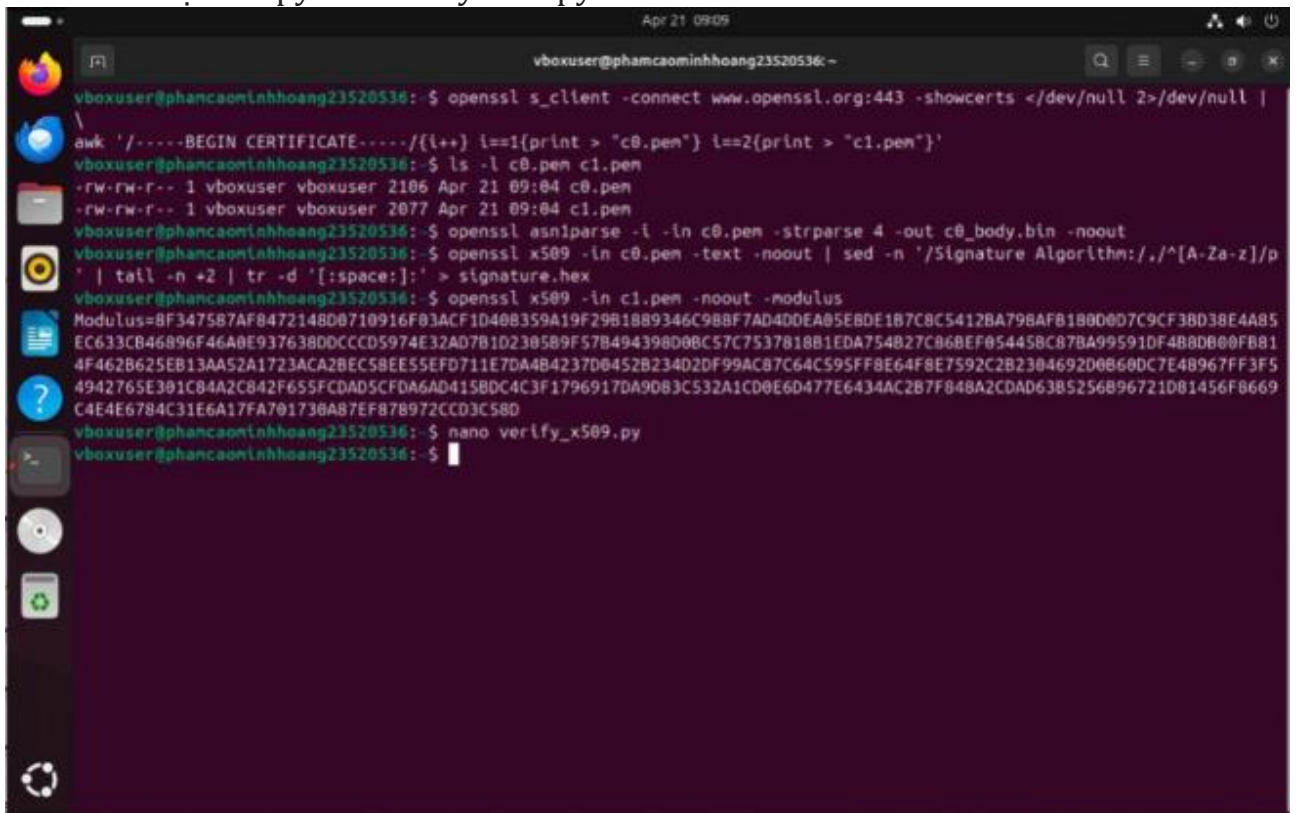
- B2: Trích xuất public key(e, n) từ chứng chỉ của nhà phát hành

Chữ ký số (Signature):

- Sử dụng kết hợp lệnh openssl x509, sed, tail và tr -d '[:space:]' để tự động dò tìm khối "Signature Algorithm", cắt lấy các byte chữ ký, xóa bỏ khoảng trắng và dấu hai chấm .
- Kết quả thu được là một chuỗi Hex dài liền mạch, xuất ra file signature.hex.

```
HD user@phancaominhhoang23520536: $ openssl x509 -in c0.pem -noout -text | \
awk '/Signature Value:./,/^[/ ]/' | \
tail -n +2 | \
tr -d '[:space:]' > signature.hex
vboxuser@phancaominhhoang23520536: $ openssl x509 -in c1.pem -noout -modulus
Modulus=8F3475B7AF8472148D0710916F03ACF1D408359A19F29B1889346C988F7AD4DDEA05E8DE1B7C8C5412BA798AFB180D0D7C9CF3BD38E4A85
EC633CB46896F46A0E937638DDCCD5974E32AD7B1D2305B9F57B494398D08C57C7537818B1EDA754B27C86BEF05445BC87BA99591DF4B8D080FB81
4F462B625EB13AA52A1723ACA2BEC58EE55EFD711E7DA4B4237D0452B234D2DF99AC87C64C595FF8E64F8E7592C2B2304692D0B60DC7E48967FF3F5
4942765E301C84A2C842F655FCDAD5CFDA6AD415BDC4C3F1796917DA90B3C532A1CD0E6D477E6434AC2B7F848A2CDAD63B5256B96721D81456F8669
C4E4E6784C31E6A17FA701730A87EF87B972CCD3C58D
```

- B3: Tạo file python verifyx509.py:



```
vboxuser@phancaominhhoang23520536: $ openssl s_client -connect www.openssl.org:443 -showcerts </dev/null 2>/dev/null |
awk '/-----BEGIN CERTIFICATE-----/{i++} i==1{print > "c0.pem"} i==2{print > "c1.pem"}'
vboxuser@phancaominhhoang23520536: $ ls -l c0.pem c1.pem
-rw-rw-r-- 1 vboxuser vboxuser 2106 Apr 21 09:04 c0.pem
-rw-rw-r-- 1 vboxuser vboxuser 2077 Apr 21 09:04 c1.pem
vboxuser@phancaominhhoang23520536: $ openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout
vboxuser@phancaominhhoang23520536: $ openssl x509 -in c0.pem -text -noout | sed -n '/Signature Algorithm:./,/^[/A-Za-z]/p' | tail -n +2 | tr -d '[:space:]' > signature.hex
vboxuser@phancaominhhoang23520536: $ openssl x509 -in c1.pem -noout -modulus
Modulus=8F3475B7AF8472148D0710916F03ACF1D408359A19F29B1889346C988F7AD4DDEA05E8DE1B7C8C5412BA798AFB180D0D7C9CF3BD38E4A85
EC633CB46896F46A0E937638DDCCD5974E32AD7B1D2305B9F57B494398D08C57C7537818B1EDA754B27C86BEF05445BC87BA99591DF4B8D080FB81
4F462B625EB13AA52A1723ACA2BEC58EE55EFD711E7DA4B4237D0452B234D2DF99AC87C64C595FF8E64F8E7592C2B2304692D0B60DC7E48967FF3F5
4942765E301C84A2C842F655FCDAD5CFDA6AD415BDC4C3F1796917DA90B3C532A1CD0E6D477E6434AC2B7F848A2CDAD63B5256B96721D81456F8669
C4E4E6784C31E6A17FA701730A87EF87B972CCD3C58D
vboxuser@phancaominhhoang23520536: $ nano verify_x509.py
vboxuser@phancaominhhoang23520536: $
```

- Sau khi tạo được file có code sau:

```
import hashlib
import binascii

def bytes_to_int(b):
    return int.from_bytes(b, 'big')

def int_to_bytes(i, length=None):
    if length is None:
        length = (i.bit_length() + 7) // 8
    return i.to_bytes(length, 'big')
```



```

BODY_FILE = "c0_body.bin"
SIG_FILE = "signature.hex"
E = 65537

N_HEX =
"8F347587AF8472148D0710916F03ACF1D408359A19F29B1889346C988F7AD
4DDEA05E8DE1B7C8C5412BA798AFB180D0D7C9CF3BD38E4A>

with open(BODY_FILE, "rb") as f:
    tbs = f.read()

computed_hash = hashlib.sha256(tbs).digest()

with open(SIG_FILE, "r") as f:
    sig_hex = f.read().strip()
    signature = binascii.unhexlify(sig_hex)
    n = int(N_HEX, 16)

em = pow(bytes_to_int(signature), E, n)
em_bytes = int_to_bytes(em, (n.bit_length() + 7) // 8)

extracted_hash = em_bytes[-32:]

print("Hash tính từ TBS      :", binascii.hexlify(computed_hash).decode())
print("Hash giải mã từ chữ ký  :", binascii.hexlify(extracted_hash).decode())

if computed_hash == extracted_hash:
    print("✅ CHỨNG CHỈ HỢP LỆ - Xác minh thành công!")
else:
    print("❌ Chữ ký KHÔNG hợp lệ!")

```

Phân tích:

1. Xử lý định dạng dữ liệu (Helper Functions)

- Định nghĩa hai hàm `bytes_to_int` và `int_to_bytes` để chuyển đổi qua lại giữa mảng byte (dữ liệu thô) và số nguyên lớn (Integer). Đây là bước bắt buộc vì các phép toán mật mã RSA trong Python yêu cầu tính toán trên số nguyên.

2. Tính toán Mã băm độc lập (Computed Hash)

- Đọc dữ liệu nhị phân từ tệp phần thân chứng chỉ (`c0_body.bin`).
- Dùng `hashlib.sha256()` để băm dữ liệu này thành một đoạn mã băm dài 32 bytes (`computed_hash`). Đây là dữ liệu đại diện cho nội dung chứng chỉ hiện tại.

3. Giải mã Chữ ký số (RSA Decryption)

- Nạp chuỗi chữ ký (signature.hex), Modulus (\$n\$) và Exponent (\$E=65537\$) của CA.
- Thực thi công thức giải mã RSA cốt lõi: $SEM = S^E \pmod n$ thông qua hàm `pow(..., E, n)`. Phép toán này sử dụng khóa công khai của CA để "mở khóa", khôi phục lại khối dữ liệu nguyên thủy (em_bytes) mà CA đã tạo ra lúc ký.

4. Trích xuất và So khớp (Verification)

- Theo chuẩn đệm PKCS#1 v1.5, mã băm luôn nằm ở phần đuôi của khối dữ liệu giải mã. Chương trình dùng cú pháp `em_bytes[-32:]` để cắt lấy đúng 32 bytes cuối cùng (extracted_hash).
 - Dem so sánh `extracted_hash` với `computed_hash`. Nếu hai mảng byte này trùng khớp hoàn toàn, chương trình kết luận chữ ký hợp lệ (chứng chỉ không bị giả mạo).
- B4: Sau khi chạy code, ta được kết quả: Chứng chỉ hợp lệ - xác minh thành công sau khi thoả điều kiện dựa trên công thức RSA

```
HD user@phamcaominhhoang23520536:~$ openssl x509 -in c1.pem -noout -modulus
Modulus=8F347587AF8472148D0710916F03ACF1D408359A19F29B1889346C988F7AD4DDEA05E8DE1B7C8C5412BA798AFB180D0D7C9CF3BD38E4A85
EC633CB46896F46A0E937638DDCCD5974E32AD7B1D2305B9F57B494398D0BC57C7537818B1EDA754B27C86BEF05445BC87BA99591DF4B8DB00FB81
4F462B625EB13AA52A1723ACA2BEC58EE55EFD711E7DA4B4237D0452B234D2DF99AC87C64C595FF8E64F8E7592C2B2304692D0B60DC7E48967FF3F5
4942765E301C84A2C842F655FCDAD5CFDA6AD415BDC4C3F1796917DA9D83C532A1CD0E6D477E6434AC2B7F848A2CDAD63B5256B96721D81456F8669
C4E4E6784C31E6A17FA701730A87EF878972CCD3C58D
vboxuser@phamcaominhhoang23520536:~$ ls
c0_body.bin  certspem  'hoang pham'  Music  Public  verify_x509.py
c0.pem      Desktop  monhoc.sh     'phan cao minh hoang'  signature.hex  Videos
c1.pem      Documents monhoc.txt    'phan hoang'           snap
certspe:    Downloads monhoc.txt.save Pictures               Templates
vboxuser@phamcaominhhoang23520536:~$ nano verify_x509.py
vboxuser@phamcaominhhoang23520536:~$ python3 verify_x509.py
Hash tính từ TBS      : c0cf40cca46f93f25cebd141b26758a57b865d15e7f892b0c5c6240fbd4b1013
Hash giải mã từ chữ ký : c0cf40cca46f93f25cebd141b26758a57b865d15e7f892b0c5c6240fbd4b1013
✅ CHỨNG CHỈ HỢP LỆ - Xác minh thành công!
vboxuser@phamcaominhhoang23520536:~$
```

HẾT